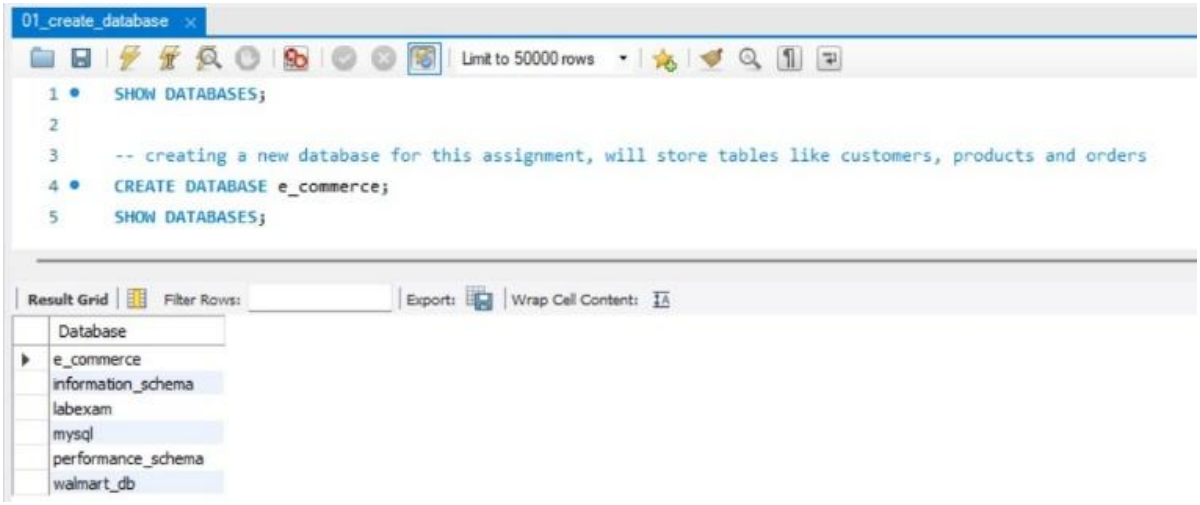
Name : More Shrinivas Balaji
Topic : Basic SQL Assignment

1)Create Database

- Created a database named e_commerce to manage all tables.



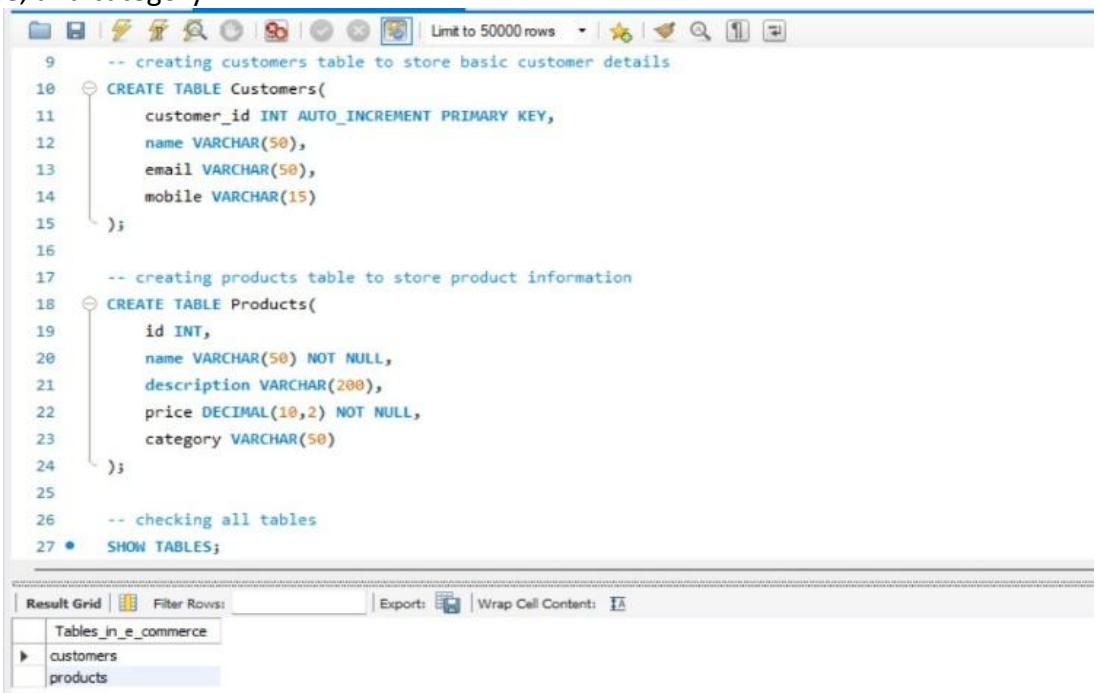
2)Table Creation

- **Customers Table**

Created the Customers table to store customer details such as name, email and mobile number.

- **Products Table**

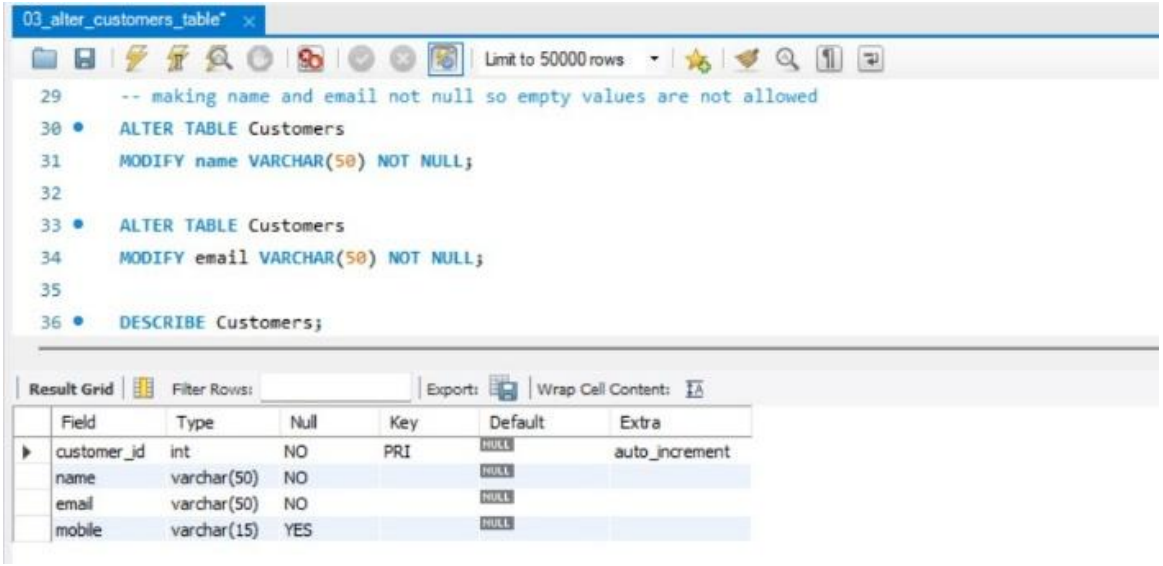
Created the Products table to maintain product information including id, name, description, price, and category.



3)Table Modifications

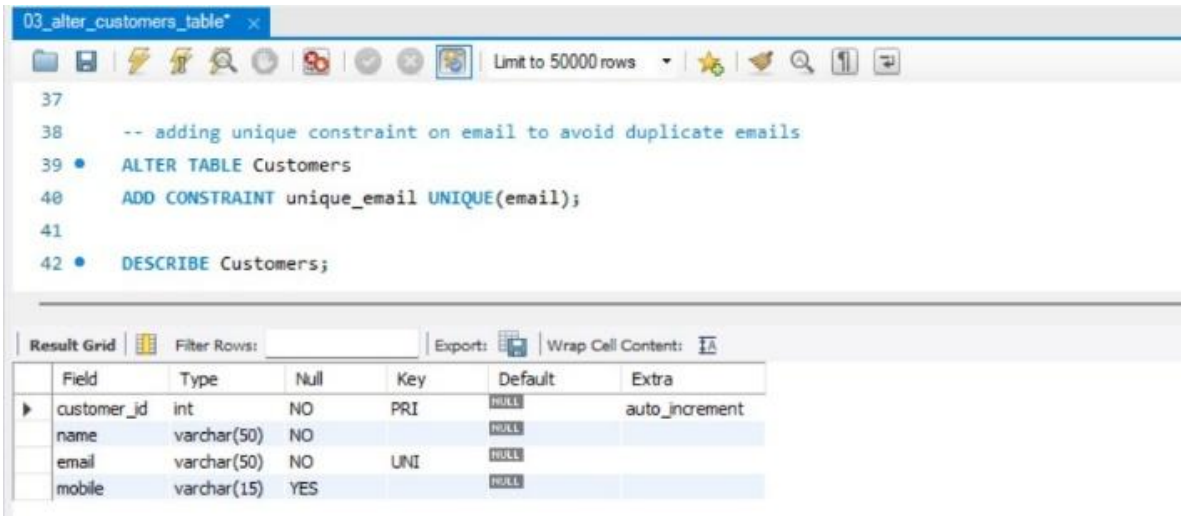
a) Add not null on name and email in the Customers table

- Applied NOT NULL constraint on name and email to ensure mandatory data entry



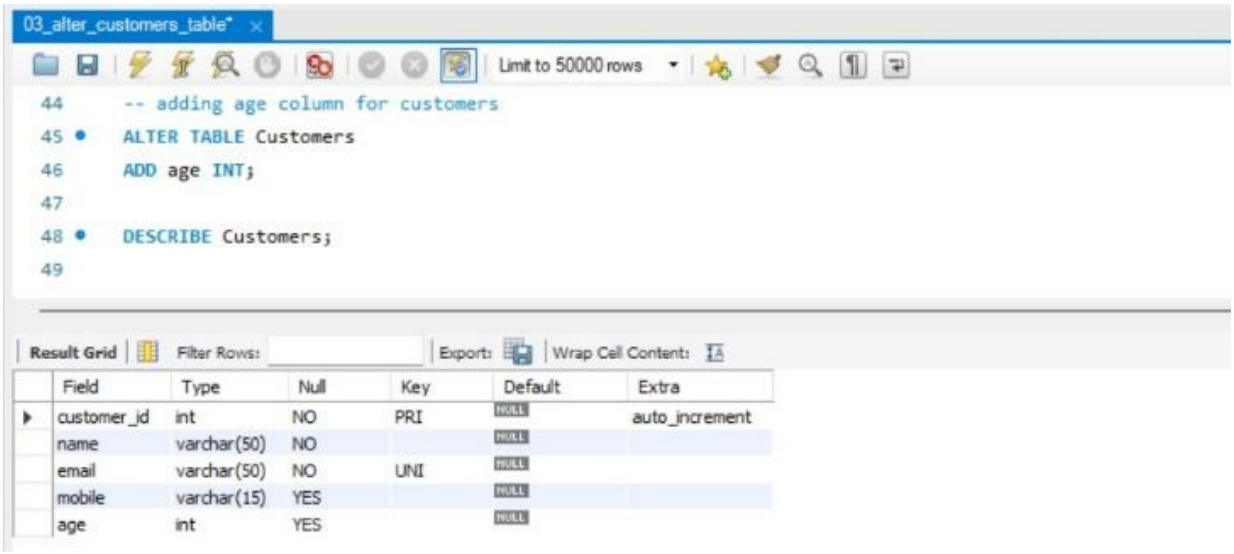
b) Add unique key on email in the Customers table

- Added UNIQUE constraint on email to prevent duplicate records.



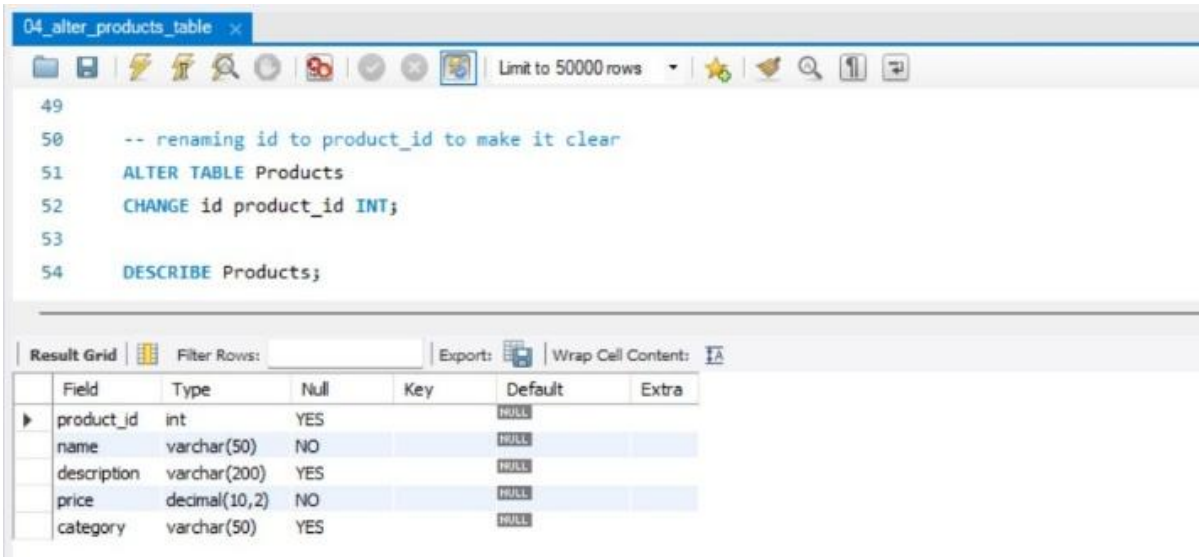
c) Add column age in the Customers table

- Added a new column ‘age’ .



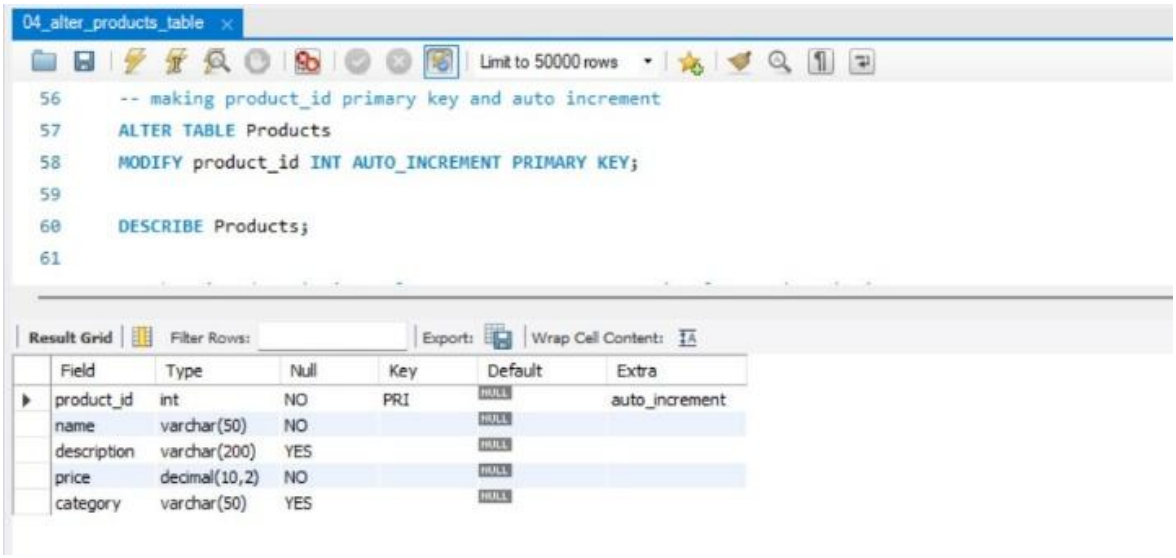
d)Change column name from id to product_id in the Products table;

- Renamed column id to product_id for better readability and consistency.



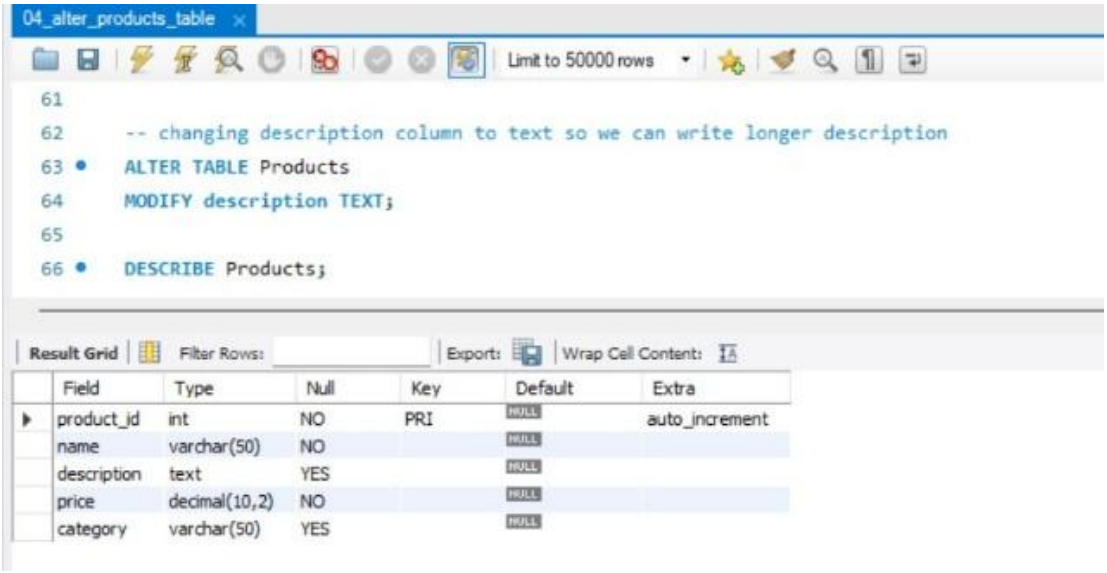
e) Add primary key and auto increment on product_id in the Products table

- Defined product_id as primary key and enabled auto increment.



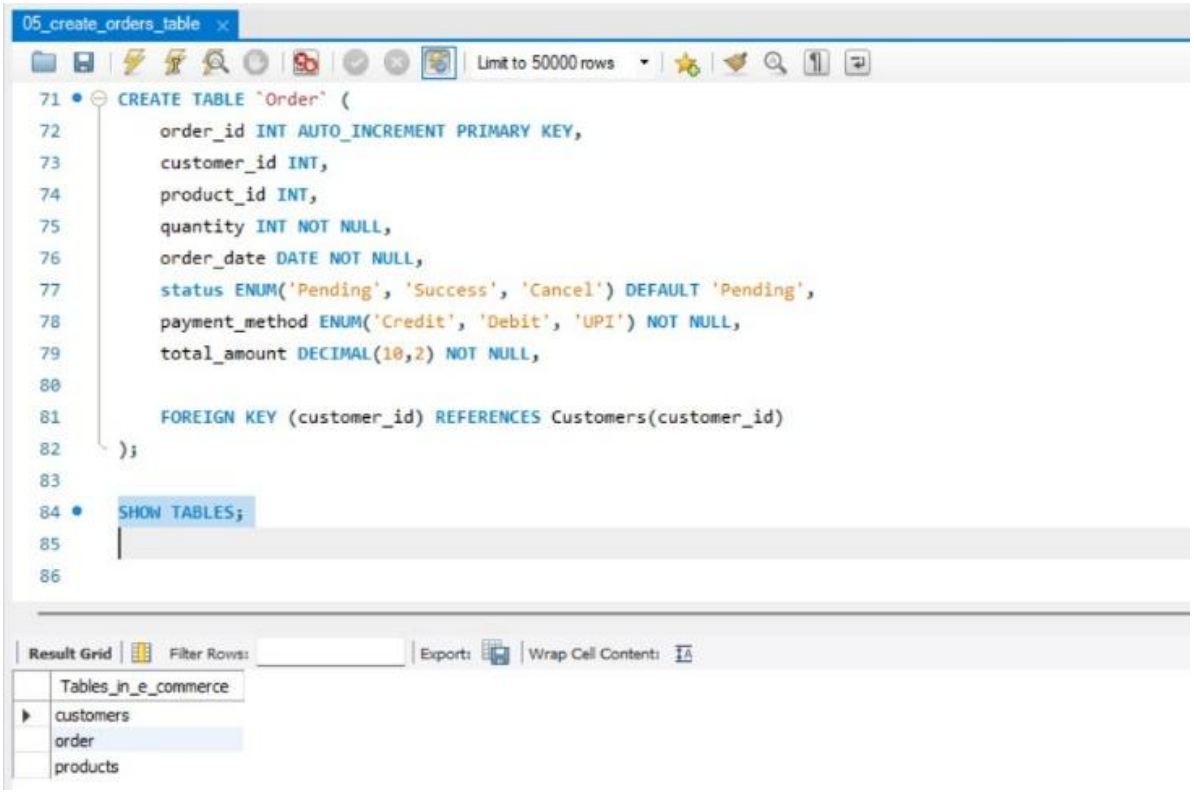
f) Change datatype of description from varchar to text in the Products table

- Modified description datatype from VARCHAR to TEXT to support larger content.



4)Orders Table Creation

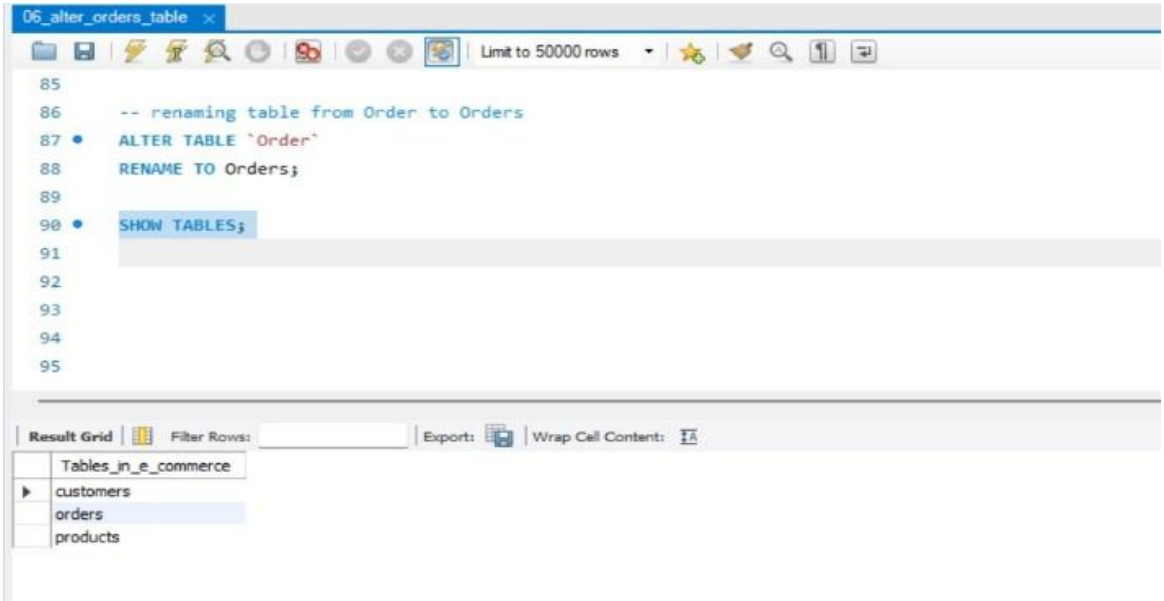
- Created Orders table to store transaction details including id,customer, product, quantity, order date,status,total_amount and payment method.



5)Orders Table Modification

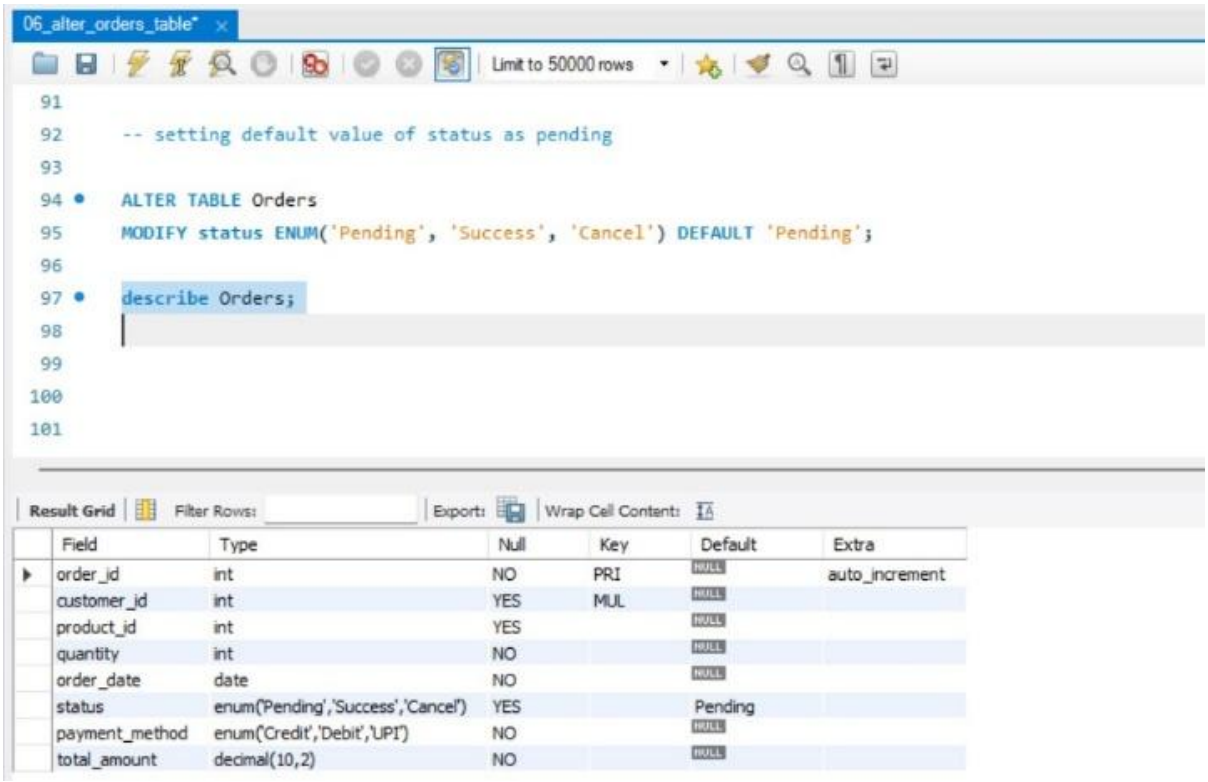
a) Change table name Order -> Orders

- Renamed table from Order to Orders for clarity.



b) Set default value pending in status.

- Set default value of status as ‘Pending’ to handle new orders.



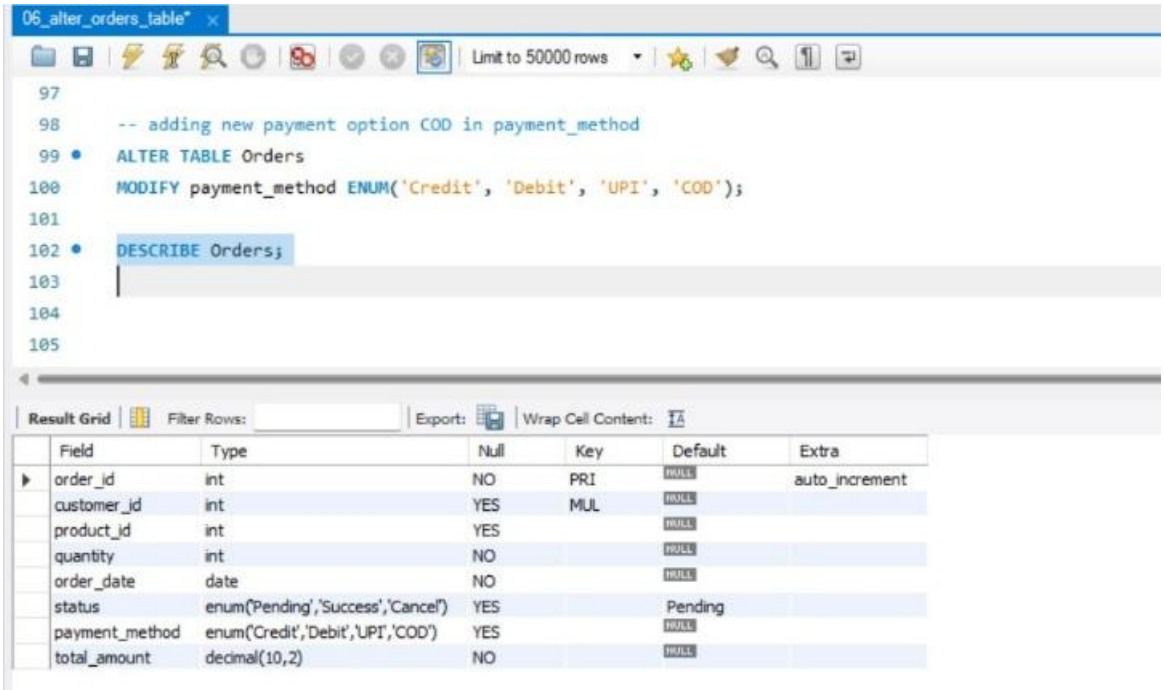
```
91
92      -- setting default value of status as pending
93
94 •    ALTER TABLE Orders
95      MODIFY status ENUM('Pending', 'Success', 'Cancel') DEFAULT 'Pending';
96
97 •    describe Orders;
98
99
100
101
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	auto_increment
customer_id	int	YES	MUL	NULL	
product_id	int	YES		NULL	
quantity	int	NO		NULL	
order_date	date	NO		NULL	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI')	NO		NULL	
total_amount	decimal(10,2)	NO		NULL	

c) Modify payment_method ENUM to add one more value: 'COD'

- Updated payment method by adding ‘COD’ option



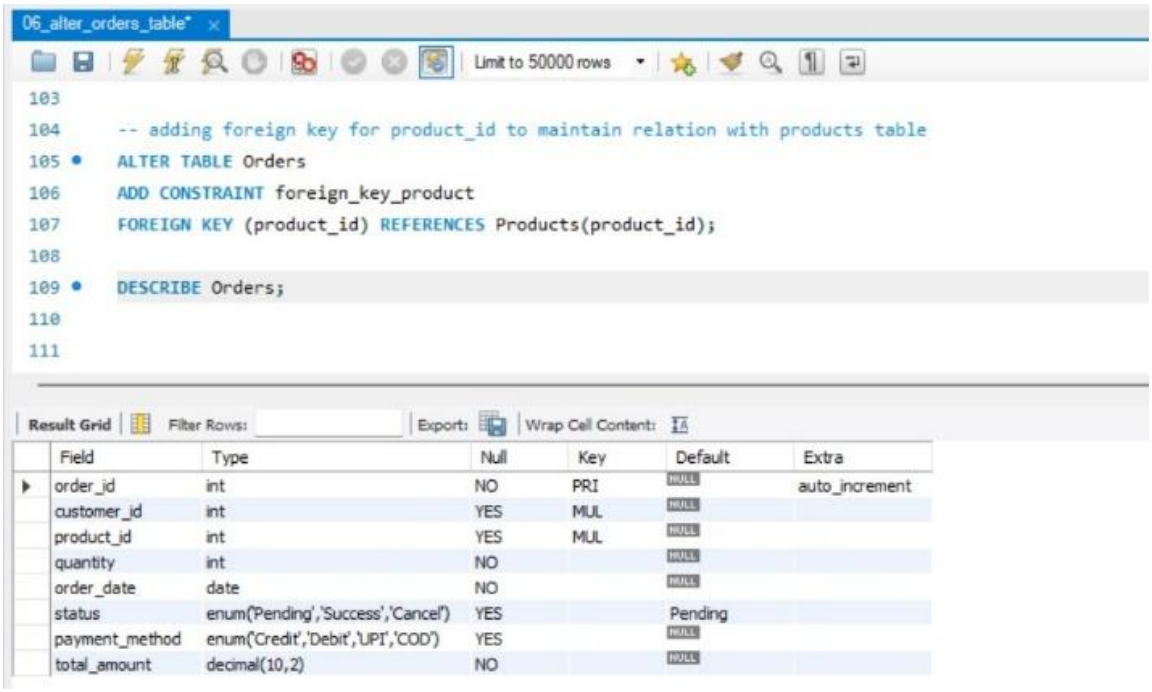
```
97
98      -- adding new payment option COD in payment_method
99 •    ALTER TABLE Orders
100     MODIFY payment_method ENUM('Credit', 'Debit', 'UPI', 'COD');
101
102 •    DESCRIBE Orders;
103
104
105
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	auto_increment
customer_id	int	YES	MUL	NULL	
product_id	int	YES		NULL	
quantity	int	NO		NULL	
order_date	date	NO		NULL	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI','COD')	YES		NULL	
total_amount	decimal(10,2)	NO		NULL	

d) Make product id as foreign key

- Established foreign key relationship for product_id to maintain data integrity.



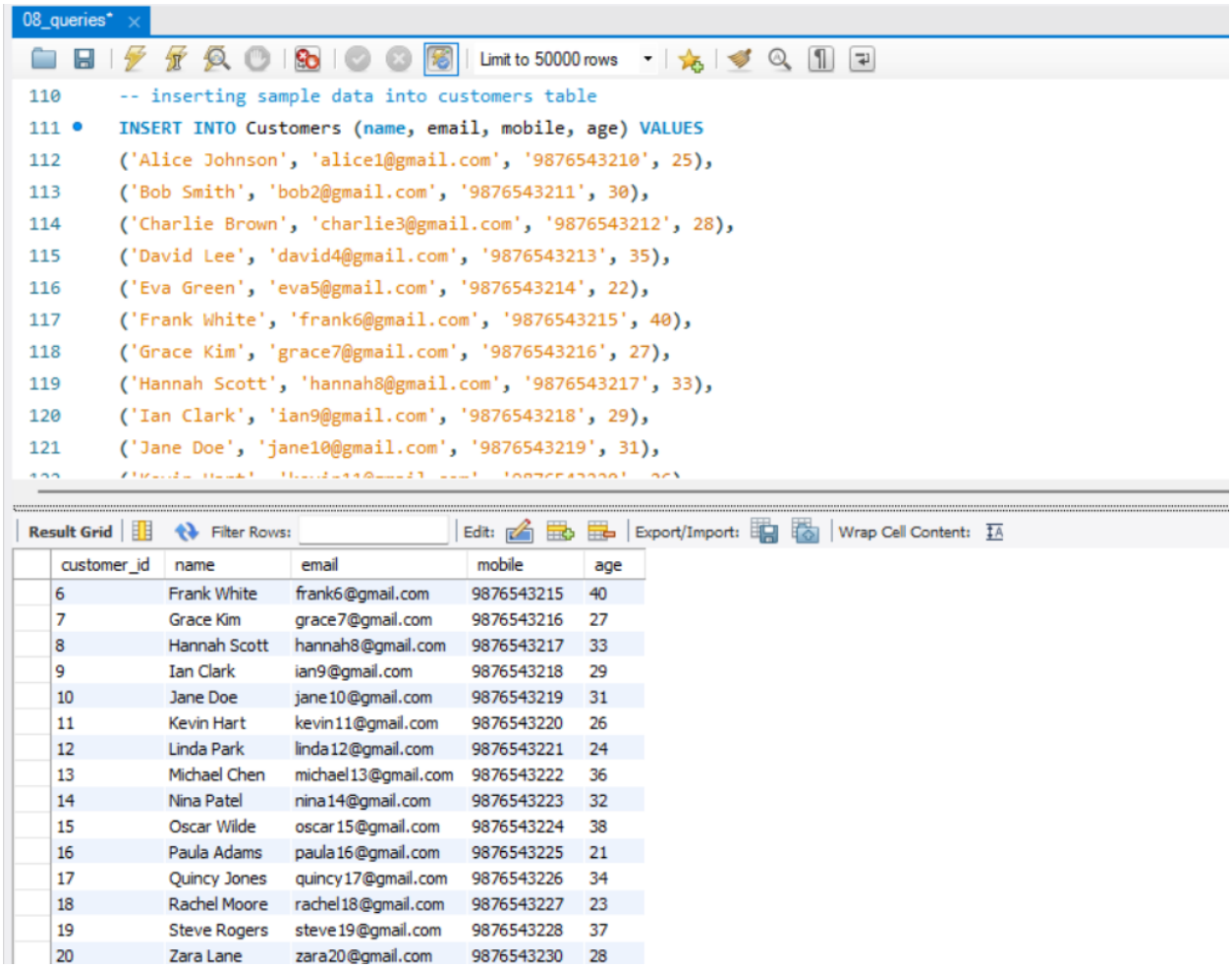
```
06_alter_orders_table*

Limit to 50000 rows

103
104     -- adding foreign key for product_id to maintain relation with products table
105 •   ALTER TABLE Orders
106     ADD CONSTRAINT foreign_key_product
107     FOREIGN KEY (product_id) REFERENCES Products(product_id);
108
109 •   DESCRIBE Orders;
110
111
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	auto_increment
customer_id	int	YES	MUL	NULL	
product_id	int	YES	MUL	NULL	
quantity	int	NO		NULL	
order_date	date	NO		NULL	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI','COD')	YES		NULL	
total_amount	decimal(10,2)	NO		NULL	

6) Insert 20 sample records in all the tables.

- Inserted sample records into all tables.



```
08_queries*
Limit to 50000 rows
110     -- inserting sample data into customers table
111 •   INSERT INTO Customers (name, email, mobile, age) VALUES
112     ('Alice Johnson', 'alice1@gmail.com', '9876543210', 25),
113     ('Bob Smith', 'bob2@gmail.com', '9876543211', 30),
114     ('Charlie Brown', 'charlie3@gmail.com', '9876543212', 28),
115     ('David Lee', 'david4@gmail.com', '9876543213', 35),
116     ('Eva Green', 'eva5@gmail.com', '9876543214', 22),
117     ('Frank White', 'frank6@gmail.com', '9876543215', 40),
118     ('Grace Kim', 'grace7@gmail.com', '9876543216', 27),
119     ('Hannah Scott', 'hannah8@gmail.com', '9876543217', 33),
120     ('Ian Clark', 'ian9@gmail.com', '9876543218', 29),
121     ('Jane Doe', 'jane10@gmail.com', '9876543219', 31),
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	mobile	age
6	Frank White	frank6@gmail.com	9876543215	40
7	Grace Kim	grace7@gmail.com	9876543216	27
8	Hannah Scott	hannah8@gmail.com	9876543217	33
9	Ian Clark	ian9@gmail.com	9876543218	29
10	Jane Doe	jane10@gmail.com	9876543219	31
11	Kevin Hart	kevin11@gmail.com	9876543220	26
12	Linda Park	linda12@gmail.com	9876543221	24
13	Michael Chen	michael13@gmail.com	9876543222	36
14	Nina Patel	nina14@gmail.com	9876543223	32
15	Oscar Wilde	oscar15@gmail.com	9876543224	38
16	Paula Adams	paula16@gmail.com	9876543225	21
17	Quincy Jones	quincy17@gmail.com	9876543226	34
18	Rachel Moore	rachel18@gmail.com	9876543227	23
19	Steve Rogers	steve19@gmail.com	9876543228	37
20	Zara Lane	zara20@gmail.com	9876543230	28

08_queries*

Limit to 50000 rows

```
134  -- inserting sample data into products table
135  INSERT INTO Products (name, description, price, category) VALUES
136  ('Laptop', 'Intel i5, 8GB RAM, 512GB SSD', 55000.00, 'Electronics'),
137  ('Smartphone', '6.5 inch display, 128GB storage', 25000.00, 'Electronics'),
138  ('Headphones', 'Wireless, Noise Cancelling', 5000.00, 'Accessories'),
139  ('Keyboard', 'Mechanical RGB keyboard', 3000.00, 'Accessories'),
140  ('Mouse', 'Wireless optical mouse', 1500.00, 'Accessories'),
141  ('Monitor', '24 inch Full HD', 10000.00, 'Electronics'),
142  ('Backpack', 'Waterproof laptop backpack', 2500.00, 'Bags'),
143  ('Chair', 'Ergonomic office chair', 7000.00, 'Furniture'),
144  ('Table', 'Wooden study table', 6000.00, 'Furniture'),
145  ('Shoes', 'Running shoes, size 9', 4000.00, 'Footwear'),
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	description	price	category
6	Monitor	24 inch Full HD	10000.00	Electronics
7	Backpack	Waterproof laptop backpack	2500.00	Bags
8	Chair	Ergonomic office chair	7000.00	Furniture
9	Table	Wooden study table	6000.00	Furniture
10	Shoes	Running shoes, size 9	4000.00	Footwear
11	Watch	Analog wrist watch	3500.00	Accessories
12	Camera	DSLR 24MP with lens	45000.00	Electronics
13	Printer	Laser printer with WiFi	12000.00	Electronics
14	Notebook	200 pages ruled notebook	200.00	Stationery
15	Pen	Gel pen pack of 5	150.00	Stationery
16	T-Shirt	Cotton round neck T-shirt	700.00	Clothing
17	Jeans	Slim fit denim jeans	1500.00	Clothing
18	Jacket	Winter jacket, waterproof	3500.00	Clothing
19	Adapter	USB-C adapter	200.00	Electronics
20	Desk Lamp	LED desk lamp	1200.00	Furniture
NULL	NULL	NULL	NULL	NULL

08_queries*

Limit to 50000 rows

```
158    -- inserting sample data into orders table
159 •  INSERT INTO Orders (customer_id, product_id, quantity, order_date, status, payment_method, total_amou
160    (1, 1, 1, '2026-03-01', 'Pending', 'UPI', 55000.00),
161    (2, 2, 2, '2026-03-02', 'Success', 'Credit', 50000.00),
162    (3, 3, 1, '2026-03-03', 'Cancel', 'Debit', 5000.00),
163    (4, 4, 1, '2026-03-04', 'Success', 'UPI', 3000.00),
164    (5, 5, 3, '2026-03-05', 'Pending', 'COD', 4500.00),
165    (6, 6, 2, '2026-03-06', 'Success', 'Credit', 20000.00),
166    (7, 7, 1, '2026-03-07', 'Pending', 'UPI', 2500.00),
167    (8, 8, 1, '2026-03-08', 'Success', 'Debit', 7000.00),
168    (9, 9, 1, '2026-03-09', 'Cancel', 'UPI', 6000.00),
169    (10, 10, 2, '2026-03-10', 'Success', 'Credit', 8000.00),
```

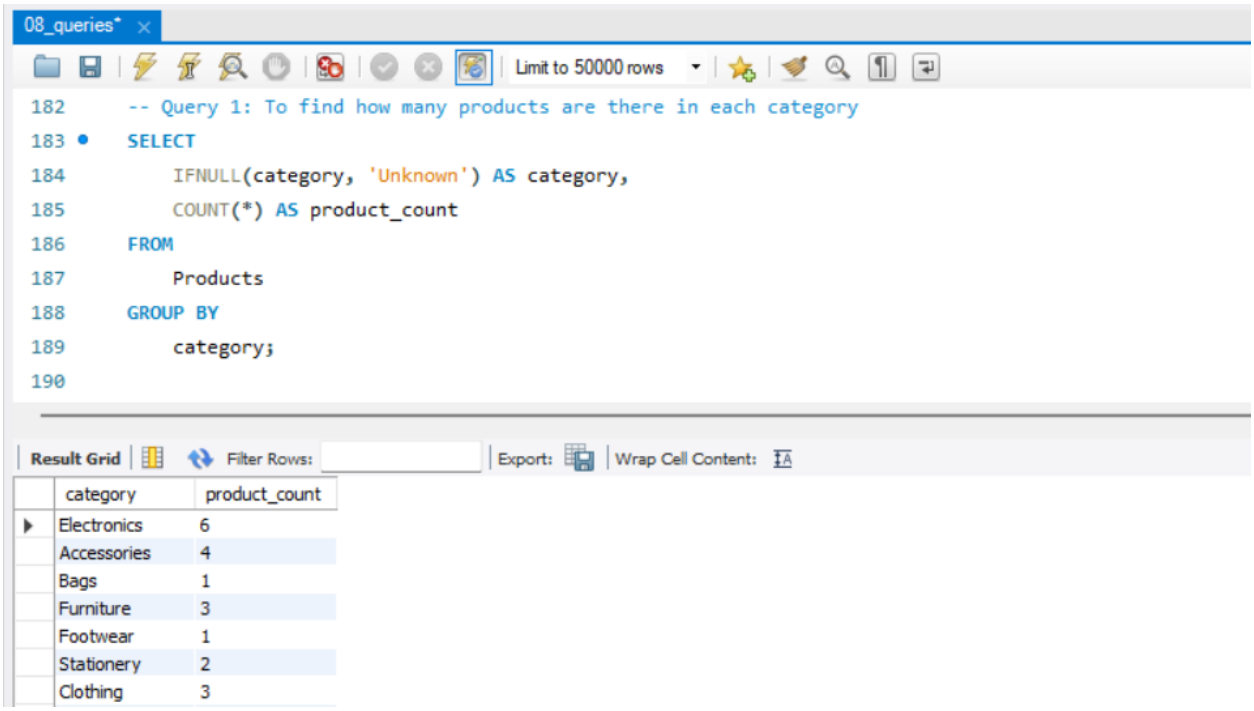
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
1	1	1	1	2026-03-01	Pending	UPI	55000.00
2	2	2	2	2026-03-02	Success	Credit	50000.00
3	3	3	1	2026-03-03	Cancel	Debit	5000.00
4	4	4	1	2026-03-04	Success	UPI	3000.00
5	5	5	3	2026-03-05	Pending	COD	4500.00
6	6	6	2	2026-03-06	Success	Credit	20000.00
7	7	7	1	2026-03-07	Pending	UPI	2500.00
8	8	8	1	2026-03-08	Success	Debit	7000.00
9	9	9	1	2026-03-09	Cancel	UPI	6000.00
10	10	10	2	2026-03-10	Success	Credit	8000.00
11	11	11	1	2026-03-11	Pending	UPI	3500.00
12	12	12	1	2026-03-12	Success	COD	45000.00
13	13	13	1	2026-03-13	Success	Credit	12000.00
14	14	14	5	2026-03-14	Pending	UPI	1000.00
15	15	15	5	2026-03-15	Success	Debit	750.00
16	16	16	3	2026-03-16	Pending	UPI	2100.00

7) Perform following queries:

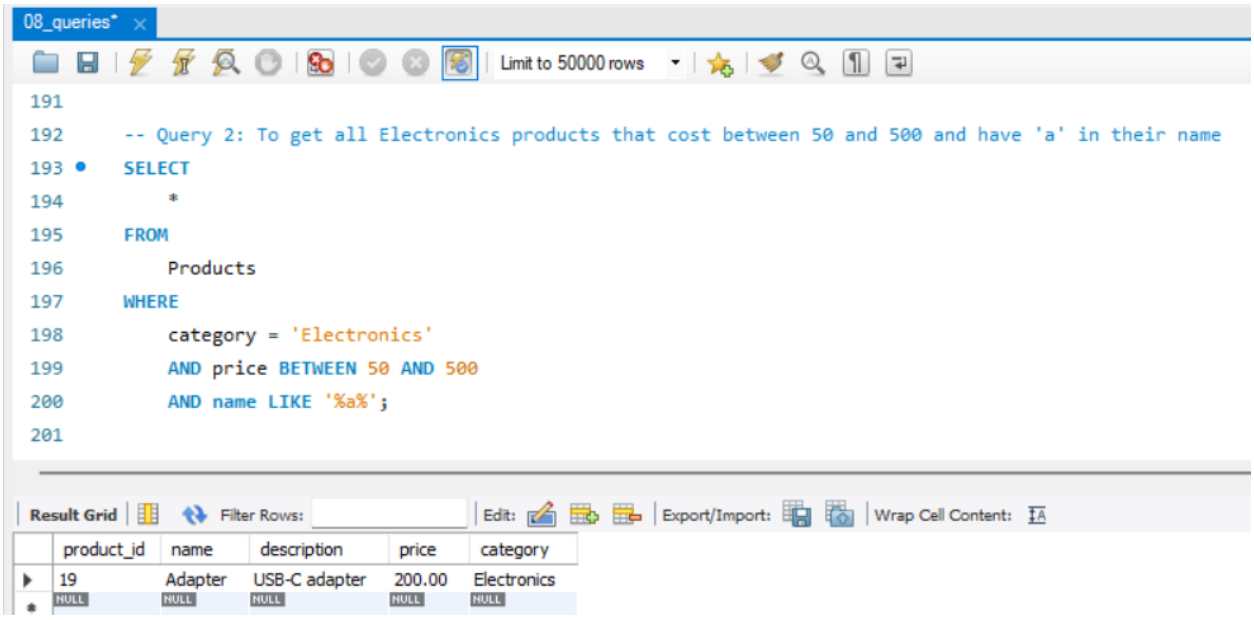
a) Count the number of products as product_count in each category.

- Used GROUP BY to group products based on category and COUNT() to get total in each group.
- Also used IFNULL so that categories with missing values are shown as ‘Unknown’ for better readability.



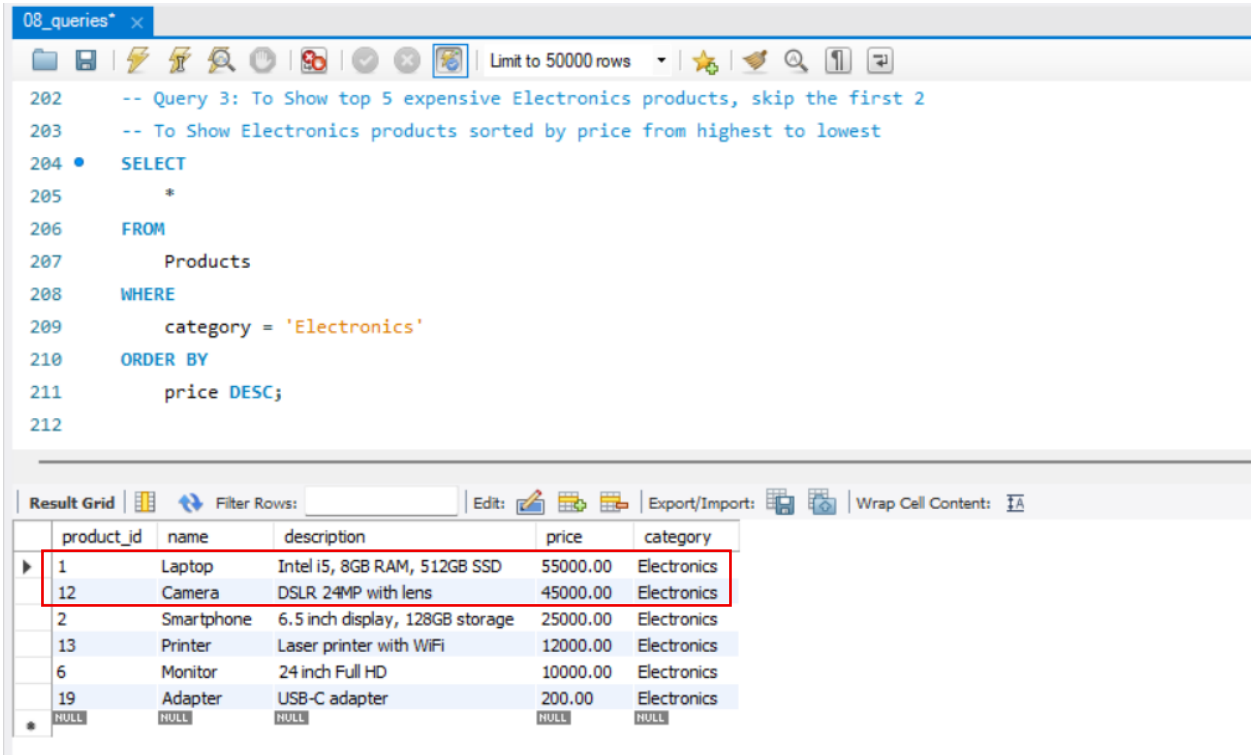
b) Retrieve all products that belong to the 'Electronics' category, have a price between $50 and $500, and whose name contains the letter 'a'.

- Used WHERE to filter only Electronics products.
- Applied BETWEEN for price range and LIKE to search products containing letter ‘a’, simulating real search conditions.

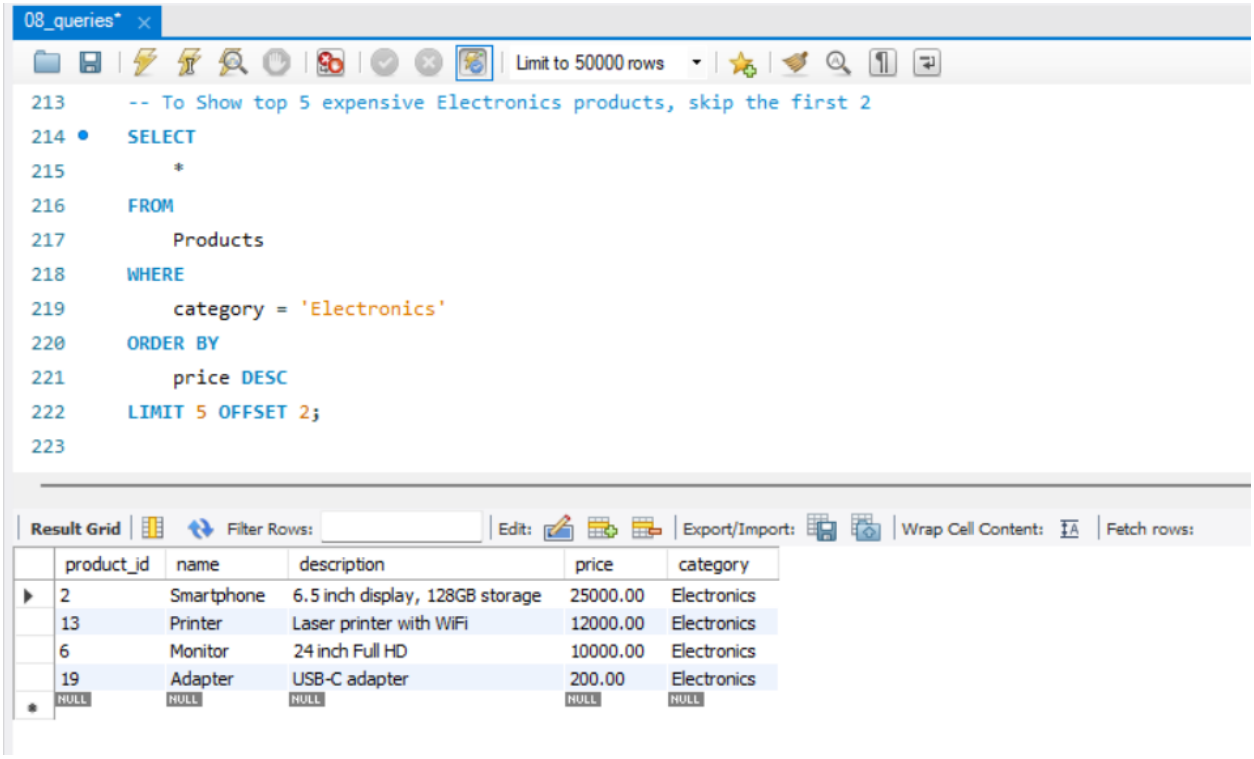


c) Get the top 5 most expensive products in the 'Electronics' category, skipping the first 2.

- Used ORDER BY DESC to sort products from highest to lowest price.

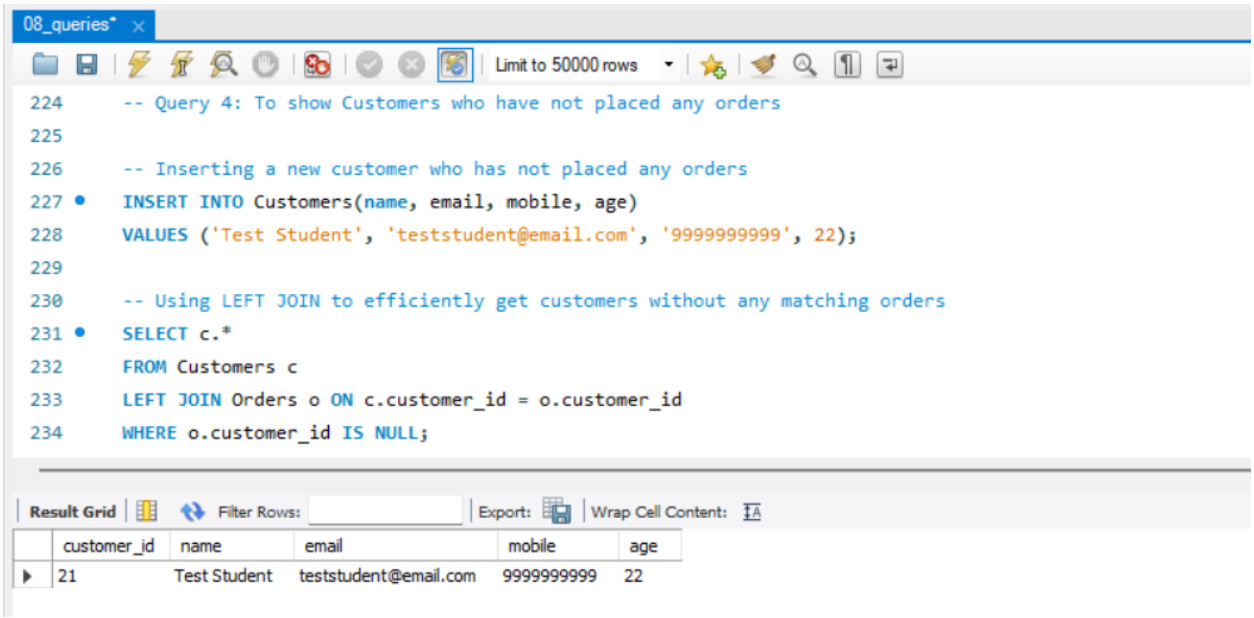


- Applied LIMIT and OFFSET to skip first few records and fetch only required top results



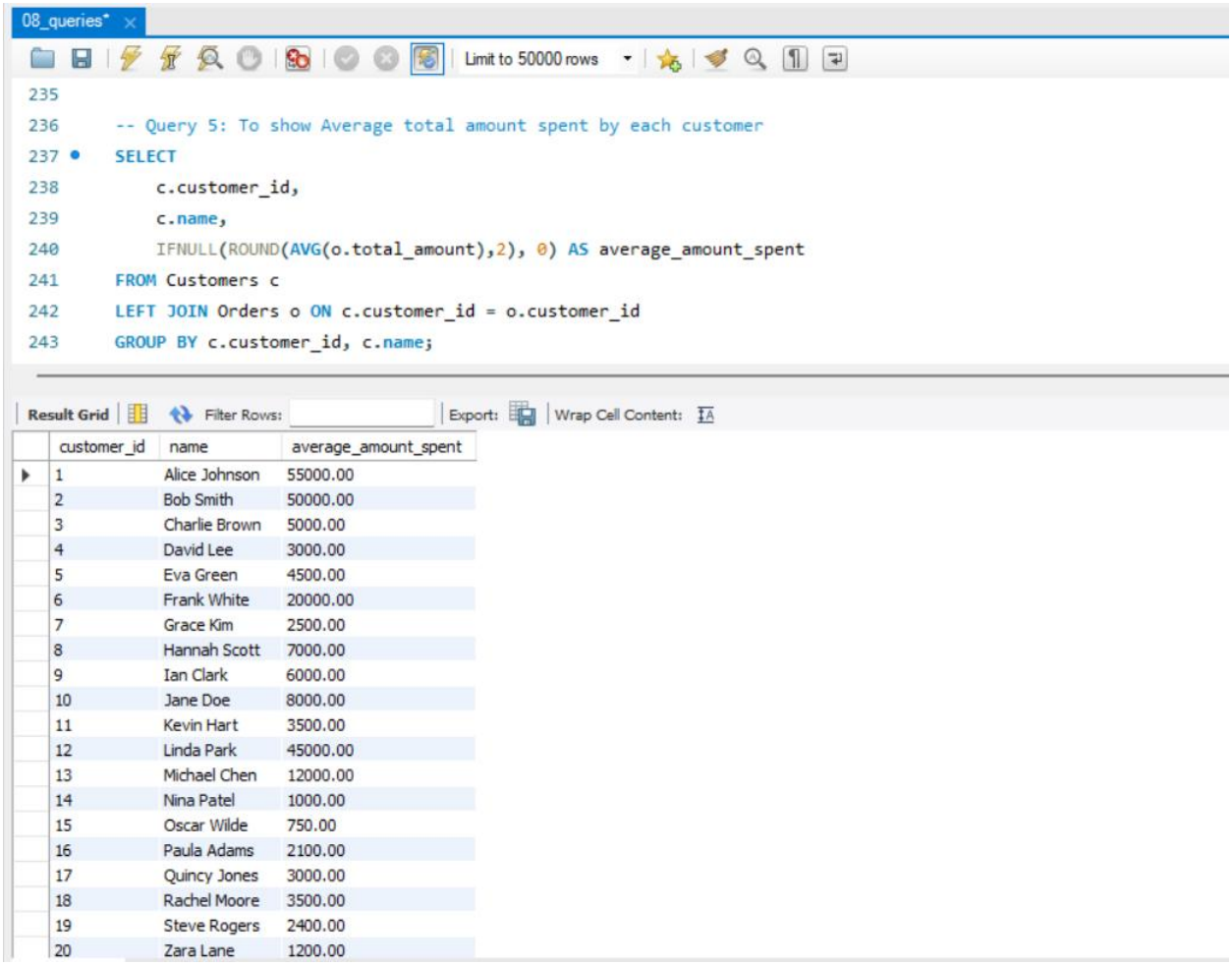
d) Retrieve customers who have not placed any orders.

- Used LEFT JOIN to include all customers even if they have no orders.
- Then filtered NULL values to find customers who have not placed any orders.
- Useful for identifying inactive users.



e) Find the average total amount spent by each customer.

- Used LEFT JOIN to include all customers in result.
- Applied AVG() to calculate average spending and IFNULL to handle customers with no orders by showing 0
- Helps in analyzing customer purchasing behavior.



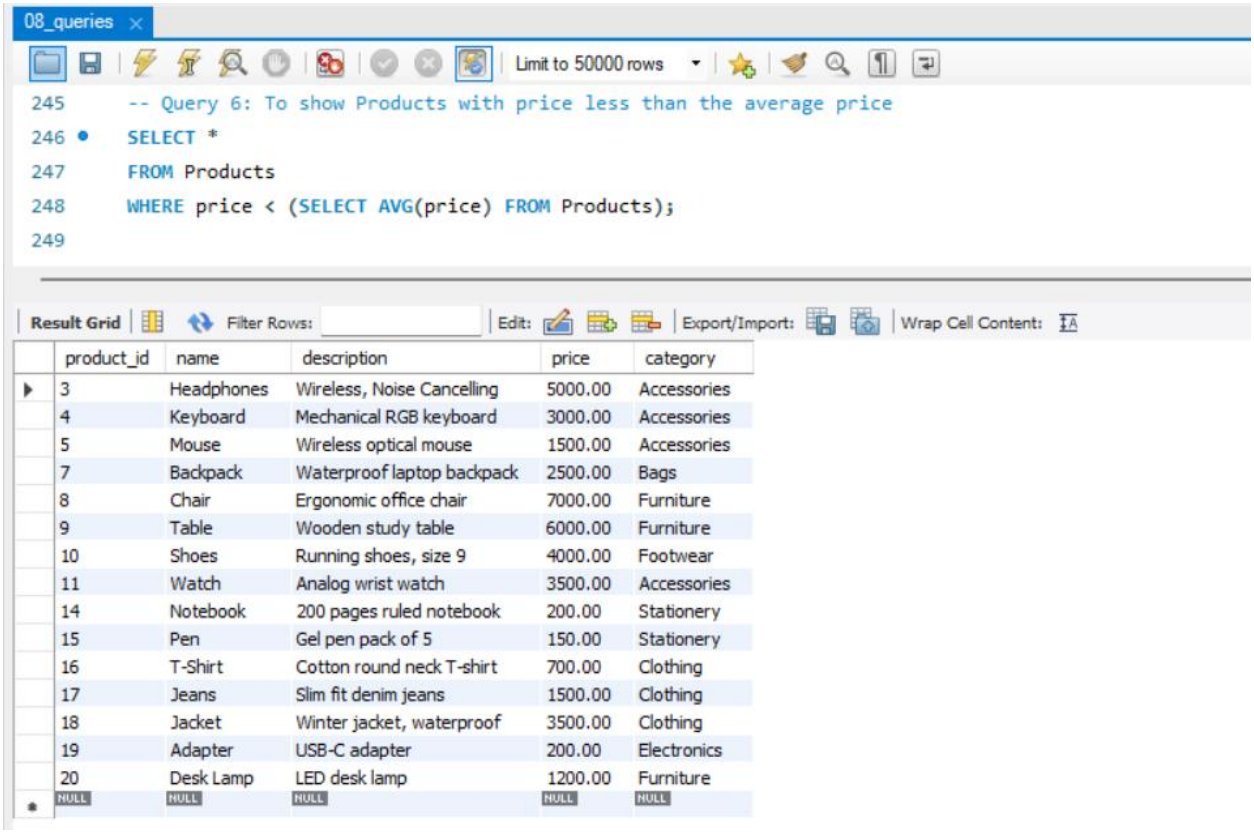
```
235
236     -- Query 5: To show Average total amount spent by each customer
237     SELECT
238         c.customer_id,
239         c.name,
240         IFNULL(ROUND(AVG(o.total_amount),2), 0) AS average_amount_spent
241     FROM Customers c
242     LEFT JOIN Orders o ON c.customer_id = o.customer_id
243     GROUP BY c.customer_id, c.name;
```

Result Grid

customer_id	name	average_amount_spent
1	Alice Johnson	55000.00
2	Bob Smith	50000.00
3	Charlie Brown	5000.00
4	David Lee	3000.00
5	Eva Green	4500.00
6	Frank White	20000.00
7	Grace Kim	2500.00
8	Hannah Scott	7000.00
9	Ian Clark	6000.00
10	Jane Doe	8000.00
11	Kevin Hart	3500.00
12	Linda Park	45000.00
13	Michael Chen	12000.00
14	Nina Patel	1000.00
15	Oscar Wilde	750.00
16	Paula Adams	2100.00
17	Quincy Jones	3000.00
18	Rachel Moore	3500.00
19	Steve Rogers	2400.00
20	Zara Lane	1200.00

f) Get the products that have a price less than the average price of all products.

- Used a subquery to calculate overall average price.
- Compared each product's price with this average to find lower-priced products.
- Useful for identifying budget-friendly products.



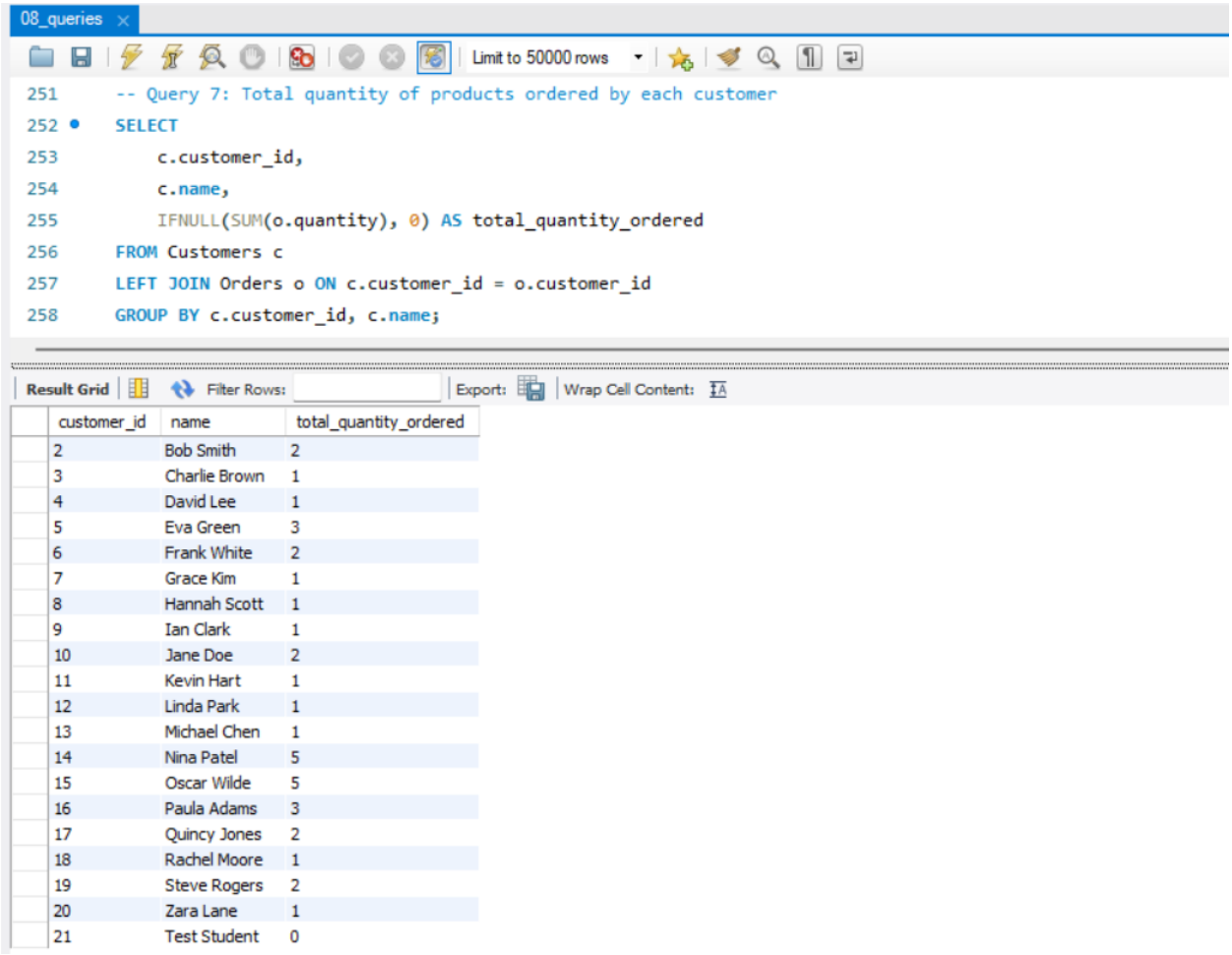
```
08_queries
Limit to 50000 rows
245     -- Query 6: To show Products with price less than the average price
246 •   SELECT *
247     FROM Products
248     WHERE price < (SELECT AVG(price) FROM Products);
249
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	description	price	category
3	Headphones	Wireless, Noise Cancelling	5000.00	Accessories
4	Keyboard	Mechanical RGB keyboard	3000.00	Accessories
5	Mouse	Wireless optical mouse	1500.00	Accessories
7	Backpack	Waterproof laptop backpack	2500.00	Bags
8	Chair	Ergonomic office chair	7000.00	Furniture
9	Table	Wooden study table	6000.00	Furniture
10	Shoes	Running shoes, size 9	4000.00	Footwear
11	Watch	Analog wrist watch	3500.00	Accessories
14	Notebook	200 pages ruled notebook	200.00	Stationery
15	Pen	Gel pen pack of 5	150.00	Stationery
16	T-Shirt	Cotton round neck T-shirt	700.00	Clothing
17	Jeans	Slim fit denim jeans	1500.00	Clothing
18	Jacket	Winter jacket, waterproof	3500.00	Clothing
19	Adapter	USB-C adapter	200.00	Electronics
20	Desk Lamp	LED desk lamp	1200.00	Furniture
NULL	NULL	NULL	NULL	NULL

g) Calculate the total quantity of products ordered by each customer:

- Used SUM() to calculate total quantity ordered.
- Used LEFT JOIN to include customers without orders and IFNULL to show 0 instead of NULL.



```
08_queries
Limit to 50000 rows
251     -- Query 7: Total quantity of products ordered by each customer
252 •   SELECT
253         c.customer_id,
254         c.name,
255         IFNULL(SUM(o.quantity), 0) AS total_quantity_ordered
256     FROM Customers c
257     LEFT JOIN Orders o ON c.customer_id = o.customer_id
258     GROUP BY c.customer_id, c.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	total_quantity_ordered
2	Bob Smith	2
3	Charlie Brown	1
4	David Lee	1
5	Eva Green	3
6	Frank White	2
7	Grace Kim	1
8	Hannah Scott	1
9	Ian Clark	1
10	Jane Doe	2
11	Kevin Hart	1
12	Linda Park	1
13	Michael Chen	1
14	Nina Patel	5
15	Oscar Wilde	5
16	Paula Adams	3
17	Quincy Jones	2
18	Rachel Moore	1
19	Steve Rogers	2
20	Zara Lane	1
21	Test Student	0

h) List all orders along with customer name and product name.

- Used INNER JOIN to combine Orders, Customers, and Products tables.
  This ensures only valid and matching records are displayed.
- Retrieved order details along with corresponding customer and product names.

08_queries*

Limit to 50000 rows

```
261    -- Query 8: To List all orders with customer and product names
262    SELECT
263        o.order_id,
264        c.name AS customer_name,
265        p.name AS product_name,
266        o.quantity,
267        o.total_amount,
268        o.status,
269        o.order_date
270    FROM Orders o
271    JOIN Customers c ON o.customer_id = c.customer_id
272    JOIN Products p ON o.product_id = p.product_id;
```

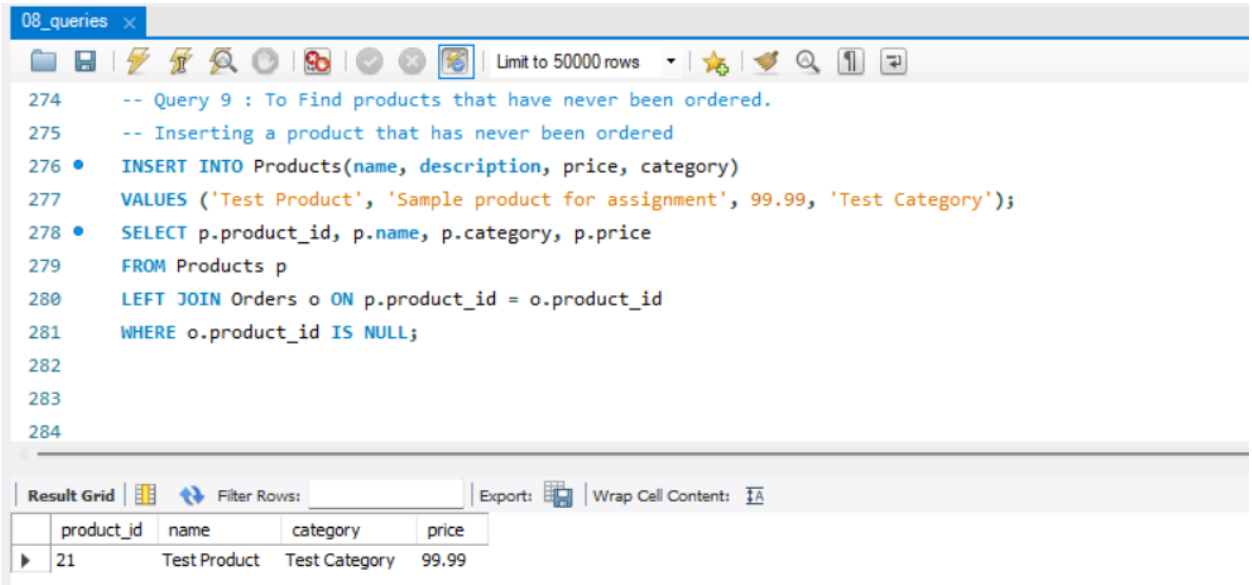
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_name	product_name	quantity	total_amount	status	order_date
1	Alice Johnson	Laptop	1	55000.00	Pending	2026-03-01
2	Bob Smith	Smartphone	2	50000.00	Success	2026-03-02
3	Charlie Brown	Headphones	1	5000.00	Cancel	2026-03-03
4	David Lee	Keyboard	1	3000.00	Success	2026-03-04
5	Eva Green	Mouse	3	4500.00	Pending	2026-03-05
6	Frank White	Monitor	2	20000.00	Success	2026-03-06
7	Grace Kim	Backpack	1	2500.00	Pending	2026-03-07
8	Hannah Scott	Chair	1	7000.00	Success	2026-03-08
9	Ian Clark	Table	1	6000.00	Cancel	2026-03-09
10	Jane Doe	Shoes	2	8000.00	Success	2026-03-10
11	Kevin Hart	Watch	1	3500.00	Pending	2026-03-11
12	Linda Park	Camera	1	45000.00	Success	2026-03-12
13	Michael Chen	Printer	1	12000.00	Success	2026-03-13
14	Nina Patel	Notebook	5	1000.00	Pending	2026-03-14
15	Oscar Wilde	Pen	5	750.00	Success	2026-03-15
16	Paula Adams	T-Shirt	3	2100.00	Pending	2026-03-16

Result 9

i)Find products that have never been ordered.

- Used LEFT JOIN to include all products.
- Filtered NULL values to find products that have no matching records in Orders.
- Helps in analyzing unsold inventory



Note: This assignment was also maintained using Git version control with a separate SQL branch and structured commits.

GitHub Repository : https://github.com/Shinu87/nucleusteq-fresher-training